ANALISI COMPARATIVA DI

MODELLI DI RACCOMANDAZIONE

Studio e Implementazione su Dataset Amazon Movies & TV

Studente	Moreno Scozzi
Dataset	Amazon Movies & TV (10.000 recensioni)
Modelli analizzati	5 (CBF, CF, Hybrid, Deep Learning)
Linguaggio	Python 3.12
Data	Marzo 2026

Presentazione del Dataset

Fonte: Amazon Product Reviews — sezione Movies and TV

Il dataset utilizzato proviene dalla piattaforma Amazon ed è stato scaricato nella versione pubblica contenente recensioni di film e serie TV. Ogni riga rappresenta la valutazione assegnata da un utente a un prodotto, accompagnata dal testo integrale della recensione. I dati sono stati preprocessati per generare due versioni distinte, utilizzate dai diversi modelli.

Struttura del Dataset

I dati grezzi vengono trasformati in due file CSV separati, ciascuno pensato per una famiglia di modelli diversa:

amazon_cf.csv — usato dai modelli di Collaborative Filtering (script 3) che lavorano solo sui rating numerici. Contiene 4 colonne: userID, itemID, rating (1.0-5.0), rating_binary (1 se rating ≥ 4.0).

Prime 10 righe di amazon_cf.csv:

userID	itemID	rating	rating_binary
A141HP4LYPWMSR	B003AI2VGA	3.0	0
A328S9RN3U5M68	B003AI2VGA	3.0	0
A1I7QGUDP043DG	B003AI2VGA	5.0	1
A1M5405JH9THP9	B003AI2VGA	3.0	0
ATXL536YX71TR	B003AI2VGA	3.0	0
A3QYDL5CDNYN66	B003AI2VGA	2.0	0
AQJVNDW6YZFQS	B003AI2VGA	1.0	0
AD4CDZK7D31XP	B00006HAXW	5.0	1
A3Q4S5DFVPB70D	B00006HAXW	5.0	1
A2P7UB02HAVEPB	B00006HAXW	5.0	1

amazon_with_text_features.csv — usato dai modelli CBF e Hybrid che sfruttano anche il testo delle recensioni. Aggiunge la colonna reviewText alle quattro precedenti.

Prime 5 righe di amazon_with_text_features.csv (testo abbreviato):

userID	itemID	rating	reviewText (troncato a 55 caratteri)
A141HP4LYPWMSR	B003AI2VGA	3.0	Synopsis: On the daily trek from Juarez, Mexico...
A328S9RN3U5M68	B003AI2VGA	3.0	THE VIRGIN OF JUAREZ is based on true events...

userID	itemID	rating	reviewText (troncato a 55 caratteri)
A1I7QGUDP043DG	B003AI2VGA	5.0	The scenes in this film can be very disquieting...
A1M5405JH9THP9	B003AI2VGA	3.0	THE VIRGIN OF JUAREZ (2006), directed by Kevin...
ATXL536YX71TR	B003AI2VGA	3.0	Informationally, this SHOWTIME original is...

Statistiche Principali

Proprietà	Valore
Righe totali (originale)	10.000
Utenti unici (originale)	8.832
Item unici (originale)	321
Righe dopo filtraggio (min. 5 interazioni)	393
Utenti dopo filtraggio	57
Item dopo filtraggio	95
Range rating	1.0 – 5.0
Rating medio	4.12
% rating >= 4.0	76,6%
Sparsità matrice user-item	92,7%

Il filtraggio riduce gli utenti da 8.832 a soli 57: necessario per avere dati sufficienti per utente, ma porta a una matrice molto sparsa (92,7%). Questo penalizza i modelli collaborativi, che funzionano meglio con molti utenti e interazioni sovrapposte.

Preprocessing dei Dati

Prima di poter usare qualsiasi modello, i dati grezzi di Amazon devono essere puliti e trasformati. Il file di partenza è movies.txt.gz — un archivio compresso in formato testo dove ogni recensione è rappresentata da coppie chiave:valore separate da righe vuote. Ho scritto uno script apposito (preprocess_amazon.py) per gestire questo passaggio.

1. Lettura del File Grezzo

Il parser legge il file riga per riga e ricostruisce ogni recensione come un dizionario Python:

```
def parse_text_file(path, limit):
    data, entry = [], {}
    with gzip.open(path, "rt", encoding="utf-8", errors="ignore") as f:
        for line in f:
            line = line.strip()
            if not line:                    # riga vuota = fine di una recensione
                if entry:
                    data.append(entry)
                    entry = {}
                    if len(data) >= limit:  # stop al limite richiesto
                        break
                continue
            if ":" in line:
                key, _, value = line.partition(":")
                entry[key.strip()] = value.strip()
    return pd.DataFrame(data)
```

Il parametro `limit` permette di estrarre solo le prime N recensioni, utile per generare i dataset a 10k, 40k e 100k righe senza dover rileggere il file più volte.

2. Pulizia e Trasformazione

Una volta ottenuto il DataFrame grezzo, si estraggono le colonne utili e si puliscono i tipi:

```
# Seleziono le colonne di interesse e le rinomino
df_cf = df[["review/userId", "product/productId", "review/score"]].dropna()
df_cf.columns = ["userID", "itemID", "rating"]

# Il rating nel file grezzo e una stringa, lo converto in float
df_cf["rating"] = pd.to_numeric(df_cf["rating"], errors="coerce")
df_cf = df_cf.dropna(subset=["rating"])

# rating_binary: 1 se positivo (>= 4.0), 0 altrimenti
# Soglia coerente con quella usata nelle metriche di valutazione
df_cf["rating_binary"] = (df_cf["rating"] >= 4.0).astype(int)
```

3. Filtraggio per Attività Minima

Per ridurre la sparsità e garantire valutazioni significative, vengono eliminati utenti e item con meno di 5 interazioni. Questo passaggio avviene dentro ogni script modello, in modo che il filtro si applichi coerentemente dopo il train/test split:

```
# Conta le interazioni per utente e per item
uc = df["userID"].value_counts()
ic = df["itemID"].value_counts()

# Mantieni solo chi ha almeno 5 interazioni
df = df[df["userID"].isin(uc[uc >= 5].index)]
df = df[df["itemID"].isin(ic[ic >= 5].index)]
```

```
# Se un utente ha valutato lo stesso item piu volte, calcola la media
df = df.groupby(["userID", "itemID"])["rating"].mean().reset_index()
```

4. Train/Test Split

Ogni script divide i dati in training (80%) e test (20%) con un seed fisso per la riproducibilità:

```
from sklearn.model_selection import train_test_split

train_df, test_df = train_test_split(
    df,
    test_size=0.20,
    random_state=42   # seed fisso: risultati sempre identici e comparabili
)
```

Il `random_state=42` garantisce che tutti e 5 i modelli lavorino esattamente sugli stessi dati di training e test, rendendo i confronti significativi.

5. Generazione dei Dataset Multi-Dimensione

Per il confronto su diverse dimensioni descritto a fine documento, il preprocess viene eseguito su tre soglie diverse. Il file viene letto una volta sola e poi suddiviso in subset:

```
SIZES = [10_000, 40_000, 100_000]

# Lettura unica fino al massimo necessario
df_raw = parse_text_file("movies.txt.gz", limit=max(SIZES))

# Per ogni dimensione: subset, preprocessing, salvataggio
for size in SIZES:
    label  = f"{size // 1000}k"
    subset = df_raw.head(size)
    df_cf, df_text = preprocess(subset)
    df_cf.to_csv(f"amazon_cf_{label}.csv",         index=False)
    df_text.to_csv(f"amazon_with_text_{label}.csv", index=False)
```

Tool 1: Scikit-Learn — Content-Based Filtering (TF-IDF + KNN)

1. Descrizione del Tool

Campo	Dettaglio
Nome	scikit-learn
Link	https://scikit-learn.org/
Sviluppatore	scikit-learn developers, INRIA (Francia)
Versione utilizzata	1.8.0
Licenza	BSD 3-Clause (open-source)
Linguaggi supportati	Python

2. Funzionalità Principali

- **Tipo di raccomandazione:** Content-Based Filtering (CBF) — raccomanda item simili a quelli già apprezzati dall'utente, basandosi sul contenuto testuale delle recensioni.
- **TF-IDF (Term Frequency-Inverse Document Frequency):** trasforma il testo in vettori numerici. Ogni parola riceve un peso proporzionale alla sua frequenza nella recensione ma inversamente proporzionale a quanto è diffusa nel dataset. Parole rare e significative (es. "capolavoro") pesano più delle parole comuni (es. "film").
- **KNN (K-Nearest Neighbors):** trova gli item più simili usando la distanza coseno tra i vettori TF-IDF. Due item sono simili se i loro vettori puntano nella stessa direzione nello spazio vettoriale.
- **Scalabilità:** buona su dataset medio-grandi grazie a implementazioni vettoriali ottimizzate in NumPy/SciPy.

3. Requisiti Tecnici e Integrazione

- Python >= 3.7, NumPy, SciPy
- Compatibile con Pandas, Matplotlib, TensorFlow, PyTorch
- Installazione: pip install scikit-learn

4. Implementazione e Codice

Il flusso implementato è il seguente:

1. Caricamento e pulizia del dataset (rimozione duplicati, filtro attività minima)
2. Train/Test split 80/20 con random_state=42
3. Estrazione TF-IDF su tutti gli item del training set con vocabolario unico
4. Addestramento KNN con metrica coseno
5. Generazione top-K raccomandazioni per ogni utente del test set
6. Calcolo metriche: Precision@10, Recall@10, F1@10, NDCG@10, RMSE, MAE

Estrazione TF-IDF (vocabolario unico su tutti gli item):

```
vectorizer = TfidfVectorizer(stop_words="english", max_features=5000)
item_list = list(train_df["itemID"].unique())
item_text = train_df.groupby("itemID")["reviewText"].first().to_dict()
texts = [item_text.get(item, "") for item in item_list]
tfidf_matrix = vectorizer.fit_transform(texts)  # shape: (n_items, 5000)
```

Addestramento KNN e generazione raccomandazioni:

```
knn = NearestNeighbors(metric="cosine", algorithm="brute", n_neighbors=20)
knn.fit(tfidf_matrix)

# Per ogni utente, aggrega i punteggi dai suoi item valutati
pred_scores = np.zeros(len(item_list))
for rated_item, r_val in rated_items.items():
    idx = item_to_idx[rated_item]
    distances, neighbors = knn.kneighbors(tfidf_matrix[idx])
    for dist, nbr_idx in zip(distances[0], neighbors[0]):
        pred_scores[nbr_idx] += (1.0 - dist) * r_val
```

5. Qualità dei Risultati e Metriche

Metrica	Valore	Interpretazione
Precision@10	0.6930	Il 69% degli item raccomandati sono rilevanti
Recall@10	0.4915	Recupera il 49% degli item rilevanti
F1@10	0.5488	Buon bilanciamento Precision/Recall
NDCG@10	0.6894	Ottimo ordinamento degli item rilevanti
RMSE	1.0861	Errore medio di ~1 punto sul rating
MAE	0.7723	Scostamento assoluto medio di 0.77
Tempo esecuzione	0.53s	Modello molto veloce
Predizioni generate	790	10 per utente × 79 utenti test

6. Facilità d'Uso e Documentazione

- Documentazione eccellente con centinaia di esempi pratici su scikit-learn.org
- API modulare e coerente, curva di apprendimento bassa
- Community molto attiva su StackOverflow, GitHub, YouTube

7. Vantaggi e Svantaggi

Vantaggi:

- Non richiede dati di altri utenti (nessun problema cold-start per gli item)
- Velocissimo in training e predizione
- Interpretabile: si può capire perché un item è stato raccomandato
- Nessuna dipendenza esterna problematica

Svantaggi:

- Non cattura i gusti collaborativi (non sa cosa piace ad utenti simili)
- Dipende dalla qualità del testo delle recensioni
- Non personalizza in base alla storia completa dell'utente

8. Conclusioni

Scikit-Learn con TF-IDF rappresenta un punto di partenza solido e affidabile per i sistemi di raccomandazione basati su contenuto. Le performance ottenute (Precision@10 = 0.69, NDCG = 0.69) sono eccellenti considerando la dimensione ridotta del dataset. È il modello raccomandato per un primo prototipo rapido.

Tool 2: Gensim — Content-Based Filtering (Doc2Vec)

1. Descrizione del Tool

Campo	Dettaglio
Nome	Gensim
Link	https://radimrehurek.com/gensim/
Sviluppatore	Radim Rehurek
Versione utilizzata	4.3.x
Licenza	GNU LGPL (open-source)
Linguaggi supportati	Python

2. Funzionalità Principali

- **Tipo di raccomandazione:** Content-Based Filtering con rappresentazioni semantiche profonde.
- **Doc2Vec:** algoritmo che estende Word2Vec ai documenti interi. A differenza di TF-IDF che conta le parole, Doc2Vec addestra una rete neurale leggera che impara a rappresentare ogni documento come un vettore denso (embedding) in uno spazio semantico. Parole semanticamente simili (es. "buono" e "eccellente") finiscono vicine nello spazio vettoriale, anche se sono parole diverse.
- **Scalabilità:** ottimizzato per corpus di grandi dimensioni con lazy loading su disco.
- **Similarità coseno:** pre-calcola la matrice di similarità tra tutti gli item per efficienza.

3. Requisiti Tecnici e Integrazione

- Python >= 3.7, NumPy, SciPy
- Installazione: pip install gensim
- Compatibile con Pandas, scikit-learn

4. Implementazione e Codice

Il modello Doc2Vec viene addestrato sui testi delle recensioni del training set, producendo un embedding vettoriale per ogni item:

```
# Preparazione documenti taggati
tagged_docs = [
    TaggedDocument(
        words=gensim.utils.simple_preprocess(text),
        tags=[str(item_id)]
    )
    for item_id, text in zip(item_text_df["itemID"], item_text_df["reviewText"])
]

# Training Doc2Vec
model_d2v = Doc2Vec(vector_size=100, window=5, min_count=1, epochs=40)
model_d2v.build_vocab(tagged_docs)
model_d2v.train(tagged_docs, total_examples=model_d2v.corpus_count,
```

```
                epochs=model_d2v.epochs)

# Pre-calcolo similarità coseno tra tutti gli item
item_vectors = np.array([model_d2v.dv[str(item)] for item in item_list])
sim_matrix = cosine_similarity(item_vectors)  # (n_items x n_items)
```

5. Qualità dei Risultati e Metriche

Metrica	Valore	Interpretazione
Precision@10	0.6953	Lievemente superiore a Sklearn
Recall@10	0.4938	Recupero simile a Sklearn
F1@10	0.5512	Migliore bilanciamento P/R
NDCG@10	0.6909	Ordinamento leggermente migliore
RMSE	1.3066	Errore rating maggiore di Sklearn
MAE	0.9285	Errore assoluto maggiore
Tempo esecuzione	0.76s	Leggermente più lento di Sklearn
Predizioni generate	790	10 per utente × 79 utenti test

6. Vantaggi e Svantaggi

Vantaggi:

- Cattura la semantica profonda del testo, non solo il conteggio delle parole
- Scalabile su grandi corpus grazie al lazy loading
- Migliori performance ranking (Precision, F1) rispetto a TF-IDF

Svantaggi:

- RMSE e MAE peggiori di TF-IDF: meno preciso nella predizione del rating numerico
- Richiede preprocessing testuale accurato
- Risultati non deterministici senza seed fisso

7. Conclusioni

Gensim con Doc2Vec supera Scikit-Learn nelle metriche di ranking (Precision, F1, NDCG), confermando che le rappresentazioni semantiche profonde catturano meglio la somiglianza tra film. È preferibile quando la qualità dell'ordinamento è più importante della precisione numerica del rating.

Tool 3: Collaborative Filtering via NMF (scikit-learn)

1. Descrizione del Tool e Scelta Motivata

Il modello originalmente pianificato era scikit-surprise, una libreria dedicata al Collaborative Filtering con algoritmi SVD, NMF e KNN. Tuttavia, durante l'implementazione è emersa una incompatibilità critica: scikit-surprise è stato compilato con NumPy 1.x e non funziona con NumPy 2.x (versione presente nell'ambiente di sviluppo).

Scelta alternativa: Si è scelto di reimplementare il Collaborative Filtering usando NMF (Non-negative Matrix Factorization) di scikit-learn, che è l'equivalente concettuale di SVD: entrambi fattorizzano la user-item matrix in rappresentazioni latenti. Questa scelta garantisce la compatibilità e produce risultati comparabili.

Campo	Dettaglio
Algoritmo	NMF (Non-negative Matrix Factorization)
Libreria	scikit-learn 1.8.0
Equivalente concettuale	SVD di scikit-surprise
Tipo	Collaborative Filtering
Dipendenze esterne	Nessuna (solo scikit-learn)

2. Come Funziona NMF

NMF fattorizza la matrice R (utenti × item) in due matrici W (utenti × fattori latenti) e H (fattori latenti × item), tali che R ≈ W × H. I fattori latenti rappresentano concetti astratti come "film d'azione", "commedia romantica", ecc. — il modello li impara autonomamente dai dati.

```
# Costruzione user-item matrix (57 x 88)
interaction_matrix = train_df.pivot_table(
    index="userID", columns="itemID", values="rating", fill_value=0)
R = interaction_matrix.values  # sparsità: 93.7%

# NMF con 20 fattori latenti
nmf = NMF(n_components=20, max_iter=300, random_state=42, init="nndsvda")
W = nmf.fit_transform(R)    # (57 utenti, 20 fattori)
H = nmf.components_         # (20 fattori, 88 item)
R_approx = W @ H            # ricostruzione: (57, 88)
```

3. Qualità dei Risultati e Metriche

Metrica	Valore	Note
Precision@10	0.0163	Bassa — dataset troppo sparso per CF
Recall@10	0.1628	Limitato dalla sparsità
F1@10	0.0296	Performance CF limitate
NDCG@10	0.0687	Ordinamento non efficace

Metrica	Valore	Note
RMSE	3.107	Errore elevato
MAE	2.8473	Scostamento alto
Tempo esecuzione	0.08s	Estremamente veloce
Predizioni generate	430	10 per utente × 43 utenti test

Perché le performance sono basse? Con soli 57 utenti e sparsità al 93.7%, il Collaborative Filtering non ha abbastanza dati per trovare pattern collaborativi significativi. Il CF funziona bene con centinaia/migliaia di utenti con gusti sovrapposti. Con il dataset AppInventory, molto più denso, ci si aspettano performance significativamente migliori.

4. Vantaggi e Svantaggi

Vantaggi:

- Non richiede il testo delle recensioni — funziona solo con i rating
- Cattura pattern collaborativi impliciti
- Molto veloce (0.08s)
- Nessuna dipendenza esterna problematica

Svantaggi:

- Soffre molto con dataset sparsi (cold-start problem)
- Non usa informazioni testuali degli item

5. Conclusioni

Il CF tramite NMF mostra il classico limite del Collaborative Filtering su dataset piccoli e sparsi. È il modello che beneficerà di più quando verrà applicato al dataset AppInventory. La sostituzione di Surprise con NMF di sklearn è una scelta tecnicamente equivalente, motivata dalla incompatibilità di Surprise con NumPy 2.x.

Tool 4: Hybrid Model — CF + CBF (scikit-learn)

1. Descrizione del Tool e Scelta Motivata

Il modello originalmente pianificato era LightFM, una libreria ibrida che combina Collaborative Filtering con item features. Tuttavia, LightFM non è compatibile con Python 3.12 a causa di un bug nel codice Cython compilato (rimozione di ob_digit in CPython 3.12). Non esiste versione corretta disponibile.

Scelta alternativa: Si è reimplementato l'approccio ibrido usando solo scikit-learn: TruncatedSVD per il componente CF e TF-IDF + cosine similarity per il componente CBF. Lo score finale è la media pesata dei due: score = alpha × CF_score + (1-alpha) × CBF_score, con alpha=0.5.

Campo	Dettaglio
Approccio	Hybrid: CF (TruncatedSVD) + CBF (TF-IDF)
Libreria	scikit-learn 1.8.0
Equivalente concettuale	LightFM (CF + item features)
Parametro alpha	0.5 (bilanciamento CF/CBF)
Dipendenze esterne	Nessuna

2. Architettura del Modello Ibrido

```
# Componente CF: TruncatedSVD sulla user-item matrix
svd = TruncatedSVD(n_components=20, random_state=42)
U = svd.fit_transform(R)     # (n_users, 20)
Vt = svd.components_         # (20, n_items)
R_approx = U @ Vt            # ricostruzione (n_users, n_items)

# Componente CBF: TF-IDF + similarità coseno
vectorizer = TfidfVectorizer(stop_words="english", max_features=1000)
tfidf_matrix = vectorizer.fit_transform(item_texts)
sim_matrix = cosine_similarity(tfidf_matrix)

# Score ibrido finale
hybrid_score = alpha * cf_score + (1 - alpha) * cbf_score
```

3. Qualità dei Risultati e Metriche

Metrica	Valore	Note
Precision@10	0.0023	Molto bassa
Recall@10	0.0233	Bassa
F1@10	0.0042	Limitata
NDCG@10	0.7705	Ottimo ordinamento!

Metrica	Valore	Note
RMSE	1.8338	Errore rating discreto
MAE	1.6229	Scostamento medio
Tempo esecuzione	0.14s	Velocissimo
Predizioni generate	430	10 per utente × 43 utenti test

Interpretazione risultati: Il NDCG@10=0.77 è il più alto tra i modelli non-DL, suggerendo che l'ibrido ordina bene gli item rilevanti. Le basse Precision/Recall sono dovute alla sparsità: con pochissime interazioni per utente, il componente CF non riesce a personalizzare adeguatamente. Il CBF compensa parzialmente questo limite.

4. Vantaggi e Svantaggi

Vantaggi:

- Combina i punti di forza di CF e CBF
- Il parametro alpha permette di bilanciare i due approcci
- Ottimo NDCG — il ranking degli item è di buona qualità
- Nessuna dipendenza esterna

Svantaggi:

- Su dataset sparsi, il componente CF porta poco valore aggiunto
- Il valore ottimale di alpha richiede tuning

5. Conclusioni

L'approccio ibrido mostra un NDCG eccellente ma Precision/Recall basse, confermando che su dataset sparsi il componente CF fatica. Con un dataset più denso (AppInventory), ci si aspetta un netto miglioramento delle metriche Precision/Recall mantenendo l'alto NDCG.

Tool 5: TensorFlow / Keras — Deep Learning Hybrid

1. Descrizione del Tool e Scelta Motivata

Il modello originalmente pianificato usava tensorflow_recommenders (TFRS), la libreria ufficiale Google per i sistemi di raccomandazione. Tuttavia, TFRS richiede Keras 2 mentre l'ambiente ha Keras 3.x (incompatibili). Si è quindi reimplementato il modello ibrido usando solo TensorFlow/Keras puro, senza dipendenze da TFRS, ottenendo maggiore flessibilità e piena compatibilità.

Campo	Dettaglio
Libreria	TensorFlow 2.20.0 + Keras 3.x
Approccio	Deep Learning Hybrid (User Emb. + Item Emb. + TF-IDF)
Architettura	Multi-input neural network con embeddings
Loss function	Mean Squared Error (MSE)
Ottimizzatore	Adam con learning rate decay

2. Architettura della Rete Neurale

Il modello combina tre sorgenti di informazione in una rete a più strati:

```
User ID ──▶ Embedding(32) ──────────────────────┐
Item ID ──▶ Embedding(32) ──────────────────────┤
                                                ├──▶ Concat ──▶ Dense(128,relu)
Testo   ──▶ TF-IDF(1000) ──▶ Dense(32,relu) ────┘                ▶ Dropout(0.2)
                                                                  ▶ Dense(64,relu)
                                                                  ▶ Dropout(0.1)
                                                                  ▶ Dense(1)
                                                                  ▶ Clip[1.0, 5.0]
```

Codice del modello Keras:

```
# Input layers
input_user = keras.Input(shape=(1,), dtype="int32")
input_item = keras.Input(shape=(1,), dtype="int32")
input_text = keras.Input(shape=(1000,))

# Embeddings
user_emb = keras.layers.Embedding(n_users, 32)(input_user)
user_emb = keras.layers.Flatten()(user_emb)
item_emb = keras.layers.Embedding(n_items, 32)(input_item)
item_emb = keras.layers.Flatten()(item_emb)
text_proj = keras.layers.Dense(32, activation="relu")(input_text)

# Combinazione e predizione
concat = keras.layers.Concatenate()([user_emb, item_emb, text_proj])
x = keras.layers.Dense(128, activation="relu")(concat)
x = keras.layers.Dropout(0.2)(x)
x = keras.layers.Dense(64, activation="relu")(x)
output = keras.layers.Dense(1, activation="sigmoid")(x)
output_scaled = output * 4.0 + 1.0  # scala [0,1] -> [1,5]
```

3. Qualità dei Risultati e Metriche

Metrica	Valore	Interpretazione
Precision@10	0.1333	Bassa — pochi utenti in test
Recall@10	0.7692	Ottimo — il migliore tra tutti i modelli
F1@10	0.2196	Limitato da Precision bassa
NDCG@10	0.7598	Eccellente ordinamento
RMSE	0.9654	Migliore RMSE assoluto
MAE	0.7799	Ottima precisione rating
Tempo esecuzione	2.07s	Più lento ma accettabile
Predizioni generate	71	Meno utenti coperti (cold-start)

4. Vantaggi e Svantaggi

Vantaggi:

- Migliore RMSE (0.97): la predizione numerica del rating è la più accurata
- Migliore Recall@10 (0.77): trova la maggior parte degli item rilevanti
- Architettura flessibile: facile aggiungere nuove feature
- EarlyStopping previene l'overfitting automaticamente

Svantaggi:

- Richiede più tempo di training rispetto ai modelli classici
- Richiede GPU per dataset grandi (CPU usata in questo contesto)
- Cold-start: non può predire per utenti/item non visti nel training
- Necessita di tuning dei parametri (learning rate, dropout, embedding dim)

5. Conclusioni

TensorFlow/Keras ottiene il miglior RMSE e il miglior Recall, confermando che il Deep Learning è superiore nella precisione della predizione e nel recupero degli item rilevanti. È il modello più adatto a dataset grandi e con feature ricche, come sarà il dataset AppInventory.

Tabella Comparativa dei Risultati

La seguente tabella riassume i risultati di tutti e 5 i modelli sul dataset Amazon 10k, valutati con la pipeline standardizzata usata anche per il confronto multi-dimensionale: train 80%, test 20%, random_state=42, generazione top-K su tutto il catalogo degli item, soglia rilevanza rating ≥ 4.0.

Nota metodologica: I valori riportati in questa tabella provengono dalla pipeline unificata dello script 6_compare_multidim.py, che genera le top-K raccomandazioni per ogni utente considerando tutti gli item del catalogo — scenario più realistico rispetto a una valutazione solo sulle coppie presenti nel test set. Per questo motivo Precision e Recall risultano più basse rispetto a valutazioni on-test-set, mentre NDCG e RMSE rimangono confrontabili. I trend relativi tra modelli (chi è meglio di chi) rimangono identici.

Modello	P@10	R@10	F1@10	NDCG @10	RMSE	MAE	Tempo (s)
1. Sklearn TF-IDF	0.0093	0.0930	0.0169	0.0645	1.1810	0.9183	0.29
2. Gensim Doc2Vec	0.0116	0.0930	0.0204	0.0353	1.1217	0.8513	1.42
3. CF-NMF	0.0163	0.1628	0.0296	0.0575	3.0851	2.8090	0.07
4. Hybrid CF+CBF	0.0023	0.0233	0.0042	0.0116	1.8338	1.6229	0.05
5. TensorFlow DL	0.1333	0.7692	0.2196	0.7598	0.9568	0.7804	2.80

Migliori Modelli per Metrica

Metrica	Modello Migliore	Valore
Precision@10	TensorFlow DL	0.1333
Recall@10	TensorFlow DL	0.7692
F1@10	TensorFlow DL	0.2196
NDCG@10	TensorFlow DL	0.7598
RMSE (min)	TensorFlow DL	0.9568
MAE (min)	Gensim Doc2Vec	0.8513
Velocità (min tempo)	Hybrid CF+CBF	0.05s

Analisi Critica

TensorFlow DL (5) domina su tutte le metriche di ranking e predizione: miglior Precision, Recall, F1, NDCG e RMSE. Questo risultato, apparentemente controintuitivo per un dataset così piccolo, si spiega con il fatto che il modello combina embedding utente, embedding item e feature testuali — tre sorgenti di informazione complementari — in modo end-to-end, imparando rappresentazioni più ricche.

I modelli Content-Based (1 e 2) mostrano RMSE competitivi (intorno a 1.1) e sono i più veloci dopo il CF. Con un dataset più grande beneficerebbero di un vocabolario TF-IDF più ricco e di embedding Doc2Vec più stabili.

Il modello CF-NMF (3) ottiene Recall@10 = 0.163, il secondo migliore dopo TF, ma RMSE molto elevato (3.09). Con soli 57 utenti e sparsità al 92.7%, la matrice fattorizzata non ha abbastanza segnale per predire i rating in modo accurato, anche se riesce a identificare una parte degli item rilevanti.

Il modello Hybrid (4) ottiene le metriche di ranking più basse. Su dataset sparso il componente CF introduce rumore invece che segnale utile, penalizzando lo score ibrido rispetto al solo CBF.

Note Tecniche: Bug Trovati e Problemi di Compatibilità

Bug Corretti negli Script

Bug 1 — Script Sklearn: indici non allineati (critico)

Nella versione originale, l'indice della tfidf_matrix e quello della interaction_matrix non erano allineati. I K-nearest neighbor venivano mappati sull'item sbagliato, producendo raccomandazioni casuali.

Fix: creato un unico dizionario item_to_idx usato coerentemente in tutte le operazioni.

Bug 2 — Script LightFM/Sklearn: TfidfVectorizer per-item (critico)

Il TfidfVectorizer veniva istanziato e fittato separatamente per ogni singolo item. Ogni item aveva quindi un vocabolario diverso con feature incomparabili. Il risultato era che i vettori di feature non potevano essere confrontati tra loro.

Fix: il vectorizer viene fittato una sola volta su tutti gli item del training set, producendo un vocabolario globale unico.

Bug 3 — Script TensorFlow: StringLookup con interi invece di stringhe (critico)

Il codice usava StringLookup(vocabulary=list(range(N))), ovvero una lookup table con interi [0,1,...,N], ma gli itemID sono stringhe. La lookup falliva silenziosamente restituendo sempre l'indice 0 per qualsiasi input, causando un RMSE = 3867.

Fix: la vocabulary usa le stringhe reali degli itemID: StringLookup(vocabulary=item_ids). Aggiunto anche il clipping dell'output in [1.0, 5.0].

Bug 4 — Formula Precision/Recall (tutti gli script)

La formula originale richiedeva sia est >= threshold che true_r >= threshold. Questa non è la definizione standard: la Precision@K misura quanti item tra i top-K raccomandati hanno un rating vero >= soglia, indipendentemente dallo score predetto.

Fix: Precision@K = |{top-K con true_r >= 4}| / K, Recall@K = |{top-K con true_r >= 4}| / |{tutti gli item con true_r >= 4}|.

Problemi di Compatibilità delle Librerie

Libreria	Problema	Soluzione Adottata
scikit-surprise	Compilata con NumPy 1.x, incompatibile con NumPy 2.x (presente nell'ambiente)	Sostituita con NMF di scikit-learn, approccio equivalente
LightFM	Codice Cython usa ob_digit rimosso in Python 3.12. Nessuna versione corretta disponibile	Reimplementato approccio ibrido (SVD + TF-IDF) con scikit-learn puro
tensorflow-recommenders	Richiede Keras 2, ma l'ambiente ha Keras 3.x (incompatibili)	Reimplementato con TensorFlow/Keras puro, senza tensorflow_recommenders
tensorflow 2.21	tf-keras richiede TF < 2.21, causava conflitti	Fissata versione tensorflow==2.20.0

Ambiente di Sviluppo

Componente	Versione
Sistema operativo	Ubuntu Linux (Python 3.12)
Python	3.12
scikit-learn	1.8.0
gensim	4.3.x
TensorFlow	2.20.0
NumPy	2.4.3
Pandas	ultima stabile
Ambiente virtuale	venv

Analisi Multi-Dimensionale: Effetto della Dimensione del Dataset

Guardando i risultati ottenuti con i 5 modelli sul dataset da 10.000 righe, una cosa mi ha subito colpito: i modelli collaborativi e ibridi avevano performance molto basse, mentre i modelli basati sul testo (TF-IDF e Doc2Vec) se la cavavano decisamente meglio. La prima ipotesi era che questo dipendesse dai modelli stessi, ma guardando meglio i dati mi sono reso conto che il problema era probabilmente la dimensione del dataset: con soli 57 utenti dopo il filtraggio e una sparsità del 92,7%, i modelli CF non hanno semplicemente abbastanza dati per trovare pattern collaborativi.

Ho quindi deciso di ripetere tutti e 5 gli esperimenti su tre dimensioni diverse del dataset — 10k, 40k e 100k righe estratte sempre dallo stesso file — per capire se le performance cambiassero in modo significativo con più dati disponibili. I risultati sono stati interessanti e in parte sorprendenti.

Dataset Utilizzati per il Confronto

Dimensione	Righe raw	Utenti (dopo filtro min5)	Item	Utenti test (20%)
10k	10.000	~215	~95	~43
40k	40.000	~2.245	~180	~449
100k	100.000	~7.255	~250	~1.451

I valori sono derivati dai risultati effettivi: il numero di utenti test si ricava da num_predizioni / K (es. 4.490 pred / 10 = 449 utenti test a 40k). Gli utenti totali dopo filtro corrispondono a circa 5× gli utenti test (split 80/20). Il numero di utenti e item scala significativamente con la dimensione del dataset perché il filtro minimo 5 interazioni diventa meno restrittivo quando ci sono più recensioni disponibili.

Lo Script di Confronto (6_compare_multidim.py)

Per automatizzare il processo ho scritto uno script che carica i CSV preprocessati per ciascuna dimensione, esegue tutti e 5 i modelli e raccoglie i risultati in un unico DataFrame. La struttura principale è:

```
SIZES  = ["10k", "40k", "100k"]
MODELS = [
    ("1 - Sklearn TF-IDF",  run_sklearn,    "text"),
    ("2 - Gensim Doc2Vec",  run_gensim,     "text"),
    ("3 - CF NMF",          run_nmf,        "cf"),
    ("4 - Hybrid CF+CBF",   run_hybrid,     "text"),
    ("5 - TensorFlow DL",   run_tensorflow, "text"),
]

for size in SIZES:
    df = load_and_filter(path_text=f"amazon_with_text_{size}.csv")
    for model_name, model_fn, dtype in MODELS:
        result = model_fn(df)          # ogni modello ritorna un dict di metriche
        result["Dataset"] = size
        all_results.append(result)

# Salva tutto in un unico CSV
```

```
pd.DataFrame(all_results).to_csv("comparison_multidim.csv", index=False)
```

Ogni funzione run_X() segue la stessa pipeline interna: carica i dati, fa il train/test split con lo stesso random_state=42, addestra il modello, genera le top-K predizioni per ogni utente e calcola le metriche. Questo garantisce che i confronti tra dimensioni e tra modelli siano sempre consistenti.

Risultati del Confronto Multi-Dimensionale

Dataset	Modello	P@10	R@10	F1@10	NDCG@10	RMSE	Tempo (s)
10k	1 - Sklearn TF-IDF	0.0093	0.0930	0.0169	0.0645	1.1810	0.29
10k	2 - Gensim Doc2Vec	0.0116	0.0930	0.0204	0.0353	1.1217	1.42
10k	3 - CF NMF	0.0163	0.1628	0.0296	0.0575	3.0851	0.07
10k	4 - Hybrid CF+CBF	0.0023	0.0233	0.0042	0.0116	1.8338	0.05
10k	5 - TensorFlow DL	0.1333	0.7692	0.2196	0.7598	0.9568	2.80
40k	1 - Sklearn TF-IDF	0.0042	0.0401	0.0076	0.0221	1.2070	4.87
40k	2 - Gensim Doc2Vec	0.0029	0.0290	0.0053	0.0109	1.1178	8.00
40k	3 - CF NMF	0.0136	0.1336	0.0246	0.0656	3.0641	0.73
40k	4 - Hybrid CF+CBF	0.0087	0.0846	0.0157	0.0377	1.8023	0.85
40k	5 - TensorFlow DL	0.1586	0.8258	0.2523	0.7983	1.0163	4.11
100k	1 - Sklearn TF-IDF	0.0020	0.0193	0.0036	0.0081	1.1371	30.81
100k	2 - Gensim Doc2Vec	0.0021	0.0207	0.0039	0.0099	1.0913	8.25
100k	3 - CF NMF	0.0232	0.2102	0.0415	0.1308	3.0570	2.42
100k	4 - Hybrid CF+CBF	0.0165	0.1530	0.0296	0.0825	1.7938	2.07
100k	5 - TensorFlow DL	0.1914	0.8720	0.2910	0.8461	0.9820	10.96

Analisi dei Trend Osservati

TensorFlow DL — migliora costantemente con più dati

Il modello Deep Learning è chiaramente quello che beneficia di più dell'aumento del dataset. Il NDCG@10 passa da 0.76 (10k) a 0.80 (40k) fino a 0.85 (100k), e la Precision@10 da 0.13 a 0.19. Questo era atteso: le reti neurali hanno bisogno di molti esempi per imparare rappresentazioni utili. Con 10k righe il modello stava lavorando al limite minimo, mentre con 100k inizia a esprimere il suo potenziale.

CF NMF — migliora soprattutto su Recall e NDCG

Il modello collaborativo puro mostra un miglioramento visibile su NDCG (da 0.058 a 0.131) e Recall (da 0.163 a 0.210) passando da 10k a 100k. Conferma che il CF ha bisogno di più utenti per trovare somiglianze collaborative, anche se il miglioramento rimane modesto rispetto al DL — la sparsità rimane un limite strutturale anche con 100k righe.

Modelli CBF (Sklearn, Gensim) — Precision scende, RMSE stabile

Un risultato controintuitivo: Precision@10 di sklearn e gensim scende all'aumentare del dataset (da 0.009 a 0.002 per sklearn). Questo non significa che i modelli peggiorano — significa che con più utenti e item, il problema diventa più difficile: ci sono molte più combinazioni possibili e trovare esattamente i 10 item giusti per ogni utente diventa statisticamente più arduo. Il RMSE invece rimane stabile intorno a 1.1-1.2, segno che la qualità della predizione numerica del rating non peggiora.

Modello Ibrido — il componente CF diventa più utile

L'Hybrid CF+CBF mostra un miglioramento progressivo su tutte le metriche di ranking (NDCG: 0.012 → 0.038 → 0.083), confermando che il componente CF del modello diventa sempre più utile man mano che il dataset cresce. A 100k righe il Hybrid supera i CBF puri su NDCG e Recall.

Conclusione del confronto: I risultati confermano che le performance osservate con 10k righe riflettono in parte i limiti del dataset, non solo quelli dei modelli. TensorFlow DL e CF NMF mostrano miglioramenti significativi con dataset più grandi, mentre i modelli Content-Based rimangono robusti ma meno dominanti. Per una valutazione definitiva dei modelli collaborativi sarebbe ideale lavorare con dataset ancora più densi, come il dataset AppInventory.

Conclusioni Generali e Prossimi Passi

Sintesi dei Risultati

Questo studio ha implementato e confrontato 5 approcci per i sistemi di raccomandazione su un dataset Amazon Movies & TV, analizzando sia la qualità dei modelli che il loro comportamento al variare della dimensione del dataset (10k, 40k, 100k righe). I risultati principali emersi sono:

- **TensorFlow Deep Learning** è il modello più performante su tutte le metriche principali (NDCG@10 = 0.76 a 10k fino a 0.85 a 100k) e mostra il miglioramento più consistente con dataset crescenti. È il modello più scalabile e quello che trae maggior vantaggio da più dati.
- **Modelli Content-Based (Sklearn TF-IDF, Gensim Doc2Vec)** sono competitivi su RMSE (intorno a 1.1) e molto veloci in esecuzione. Non dipendendo da pattern collaborativi, soffrono meno della sparsità ma non migliorano significativamente con dataset più grandi.
- **Collaborative Filtering (NMF)** mostra un miglioramento visibile passando da 10k a 100k (NDCG da 0.058 a 0.131, Recall da 0.163 a 0.210), confermando che ha bisogno di più utenti per trovare somiglianze collaborative significative.
- **Modello Ibrido CF+CBF** migliora progressivamente con il dataset (NDCG: 0.012 → 0.038 → 0.083), confermando che la componente collaborativa diventa sempre più utile quando le interazioni aumentano.

Limitazioni del Lavoro

- **Sparsità del dataset Amazon:** anche a 100k righe la matrice user-item rimane molto sparsa. I modelli collaborativi richiederebbero dataset con interazioni più dense per esprimere pienamente il loro potenziale.
- **Librerie sostituite per incompatibilità:** scikit-surprise, LightFM e tensorflow-recommenders sono state sostituite con alternative equivalenti (NMF sklearn, SVD+TF-IDF, Keras puro) a causa di incompatibilità con NumPy 2.x e Python 3.12. I risultati sono comparabili ma non identici alle librerie originali.
- **Nessun hyperparameter tuning sistematico:** tutti i modelli usano parametri scelti manualmente. Un tuning con grid search o cross-validation potrebbe migliorare significativamente i risultati, specialmente per NMF (n_components) e TensorFlow (embedding_dim, learning_rate, dropout).
- **Esecuzione su CPU:** TensorFlow è stato eseguito senza GPU, il che limita la velocità di training. Con GPU il tempo di esecuzione si ridurrebbe di un ordine di grandezza, rendendo praticabili dataset ancora più grandi.

Prossimi Passi

7. Applicare tutte e 5 le pipeline al dataset AppInventory fornito dal professore, verificando se i trend osservati su Amazon si confermano su un dominio diverso
8. Eseguire hyperparameter tuning sistematico: n_components per NMF, alpha per il modello ibrido, embedding_dim e learning_rate per TensorFlow
9. Valutare l'aggiunta di feature categoriali (genere, anno di uscita, categoria) ai modelli ibridi, che attualmente usano solo il testo delle recensioni
10. Implementare strategie anti cold-start per utenti e item con poche interazioni, problema emerso chiaramente dall'analisi multi-dimensionale

11. Testare dataset oltre 100k righe per verificare se TensorFlow DL continua a migliorare o raggiunge uno stallo di performance

Struttura degli Script

Script	Modello	Libreria	Output CSV
0_run_all_models.py	Script master (esegue tutti)	—	comparison_all_models.csv
1_cbf_sklearn.py	TF-IDF + KNN (CBF)	scikit-learn	results_sklearn.csv
2_cbf_gensim.py	Doc2Vec (CBF)	gensim	results_gensim.csv
3_cf_surprise.py	NMF (CF)	scikit-learn	results_surprise.csv
4_hybrid_sklearn.py	SVD + TF-IDF (Hybrid)	scikit-learn	results_hybrid_sklearn.csv
5_hybrid_tensorflow.py	Deep Learning (Hybrid DL)	tensorflow/keras	results_tensorflow.csv

Tutti gli script sono configurabili tramite il dizionario CONFIG presente in ogni file.